QUESTION 1

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function (int n)
{
        int i= 1;
  int s =1;
        while(s <= n)
        {
      i++;
         s += i;
        }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

For example:

Input	Result
9	12

ALGORITHM:

```
#include <stdio.h>

int main() {
   int n, count = 2, i = 1, s = 1;

   if (scanf("%d", &n) != 1) return 0;

   while (s <= n) {
      count += 3;
      i++;
      s += i;
   }
   count++;
```

```
    printf("%d", count);
    return 0;
}
```

INPUT:
3
OUTPUT:
9

QUESTION 2

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void func(int n)
{
    if(n==1)
    {
      printf("*");
    }
    else
    {
     for(int i=1; i<=n; i++)
     {
       for(int j=1; j<=n; j++)
       {
          printf("*");
          printf("*");
          break;
       }
     }
   }
 }
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.
Input:
A positive Integer n
Output:
Print the value of the counter variable

ALGORITHM:

```
#include <stdio.h>

int main() {
    int n, count = 0;
    if (scanf("%d", &n) != 1) return 0;

    count++;
    if (n == 1) {

    } else {
        count++;
        for (int i = 1; i <= n; i++) {
            count += 5;
        }
        count++;
    }

    printf("%d", count);
    return 0;
}
```

INPUT:
9
OUTPUT:
48

QUESTION 3

Convert the following algorithm into a program and find its time complexity using counter method.

```
 Factor(num) {
{
          for (i = 1; i <= num;++i)
          {
    if (num % i== 0)
          {
         printf("%d ", i);
          }
           }
}
```

Note: No need of counter increment for declarations and scanf() and counter variable printf() statement.

Input:
A positive Integer n
Output:
Print the value of the counter variable

ALGORITHM:

```
#include <stdio.h>

int main() {
    int num, count = 0;
    if (scanf("%d", &num) != 1) return 0;

    count++;
    for (int i = 1; i <= num; ++i) {
        count += 3;
    }
    count++;

    printf("%d", count);
    return 0;
}
```

INPUT:
4
OUTPUT:
14

QUESTION 4
Convert the following algorithm into a program and find its time complexity using counter method.

```
void function(int n)
{
    int c= 0;
    for(int i=n/2; i<n; i++)
        for(int j=1; j<n; j = 2 * j)
            for(int k=1; k<n; k = k * 2)
                c++;
```

}

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:
A positive Integer n
Output:
Print the value of the counter variable

ALGORITHM:

```
#include <stdio.h>

void function(int n) {
    int c = 0;
    for(int i = n / 2; i < n; i++) {
        for(int j = 1; j < n; j = 2 * j) {
            for(int k = 1; k < n; k = k * 2) {
                c++;
            }
        }
    }
    printf("Counter value: %d\n", c);
}

int main() {
    int n = 8;
    function(n);
    return 0;
}
```

INPUT:
8
OUTPUT:
36

QUESTION 5
Convert the following algorithm into a program and find its time complexity using counter method.

```
void reverse(int n)
{
  int rev = 0, remainder;
  while (n != 0)
```

```
        {
        remainder = n % 10;
        rev = rev * 10 + remainder;
          n/= 10;

        }
print(rev);
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:
A positive Integer n
Output:
Print the value of the counter variable

ALGORITHM:

```
#include <stdio.h>

void reverse(int n) {
    int rev = 0, remainder;
    int c = 0;
    while (n != 0) {
        remainder = n % 10;
        rev = rev * 10 + remainder;
        n /= 10;
        c++;
    }
    printf("Counter value: %d\n", c);
}

int main() {
    int n = 132;
    reverse(n);
    return 0;
}
```

INPUT:
132
OUTPUT:
3